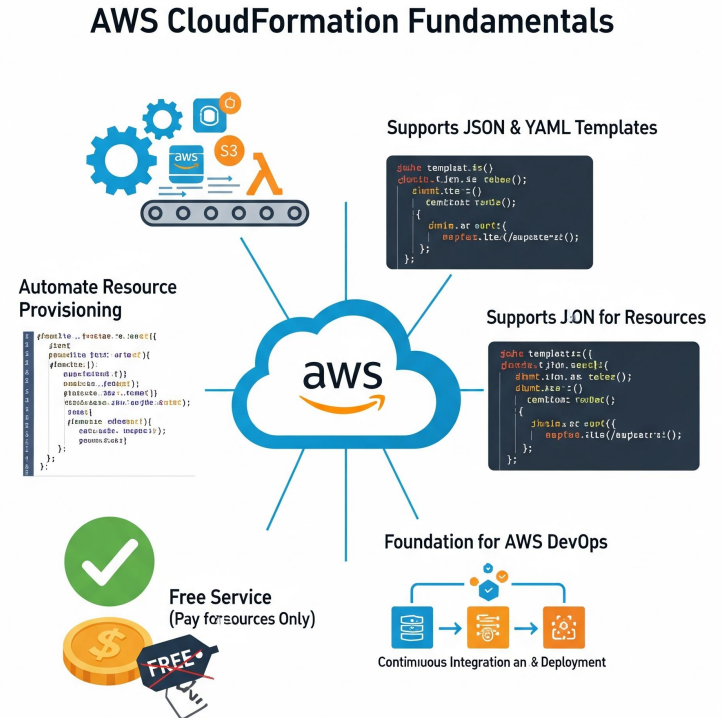


AWS Cloudformation

Core Concepts

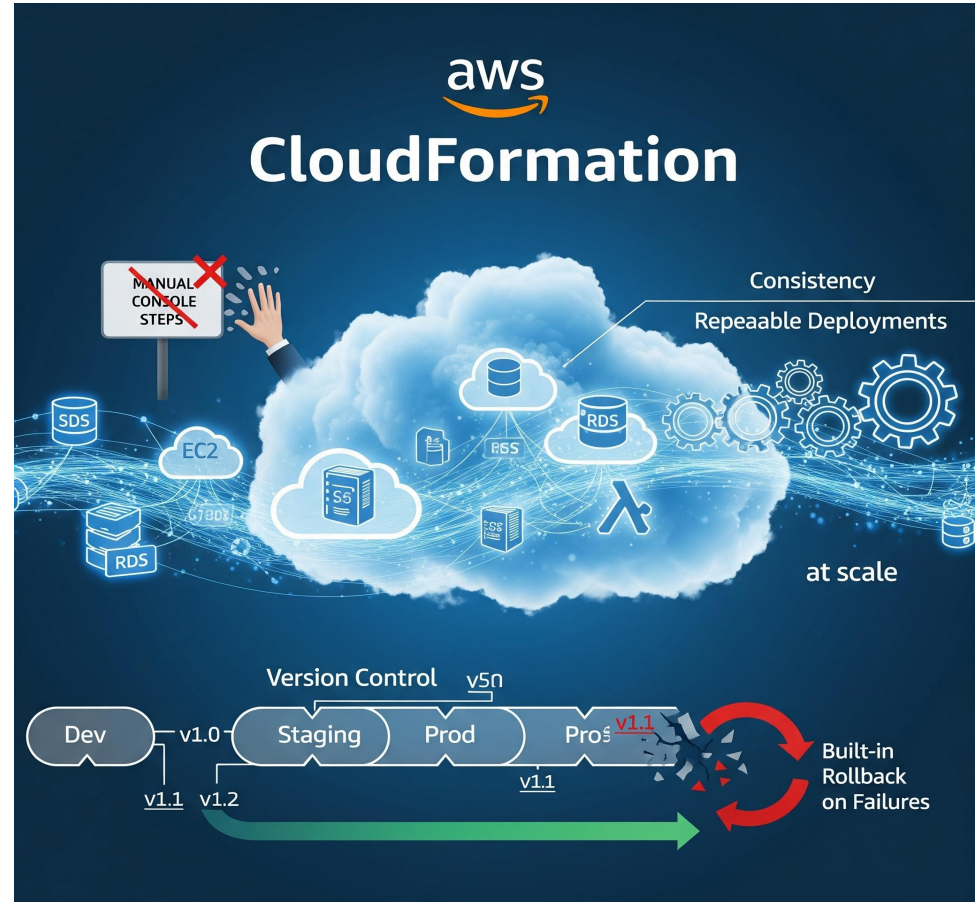
AWS Cloudformation Fundamentals

- ❖ Automate resource provisioning
- ❖ Supports JSON & YAML templates
- ❖ Free service (pay for resources only)
- ❖ Foundation for AWS DevOps



Why CloudFormation

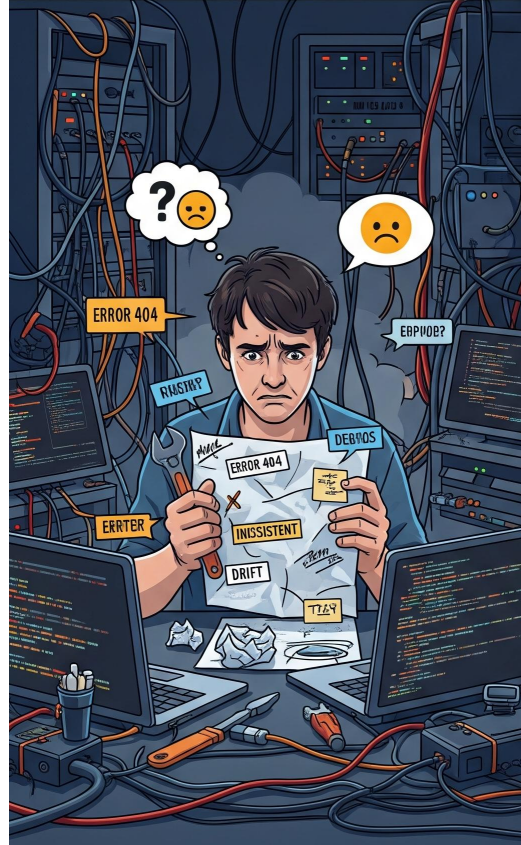
- ❖ Eliminates manual console steps
- ❖ Consistency across environments
- ❖ Version control for infrastructure
- ❖ Repeatable deployments at scale
- ❖ Built-in rollback on failures



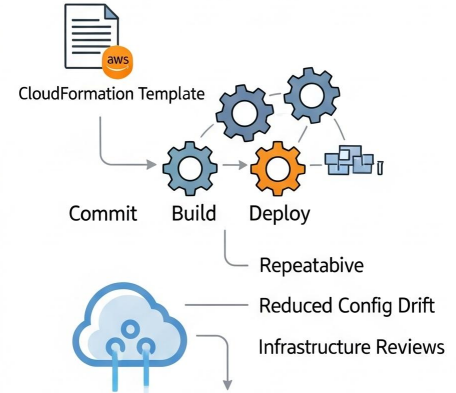
CloudFormation vs. Manu

- ❖ Manual = error-prone, inconsistent
- ❖ CFN = declarative, repeatable
- ❖ Reduces config drift
- ❖ Enables infra reviews via templates
- ❖ Integrates with CI/CD pipelines

Manual Provisioning



CloudFormation

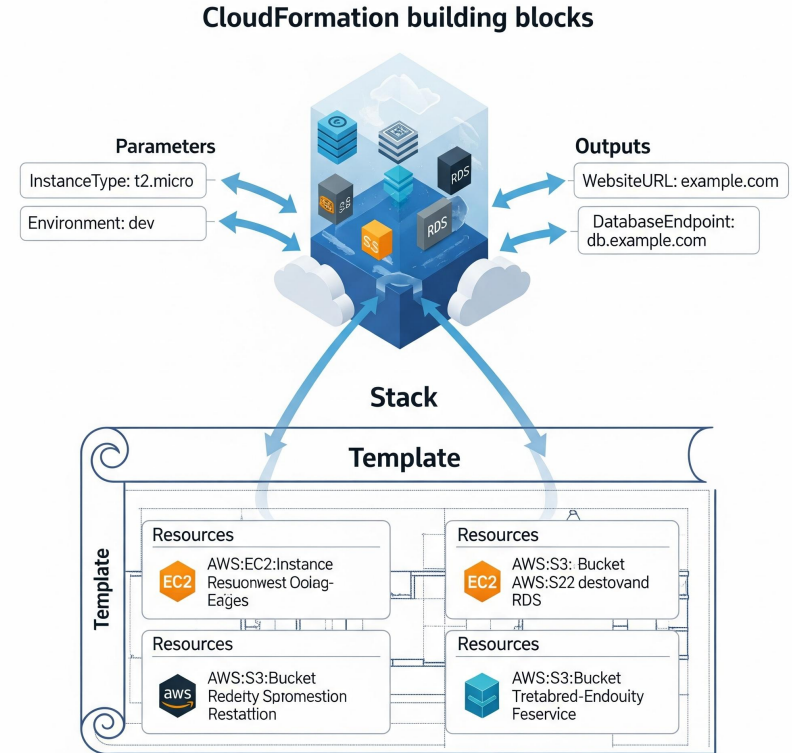


Cloud Infrastructure



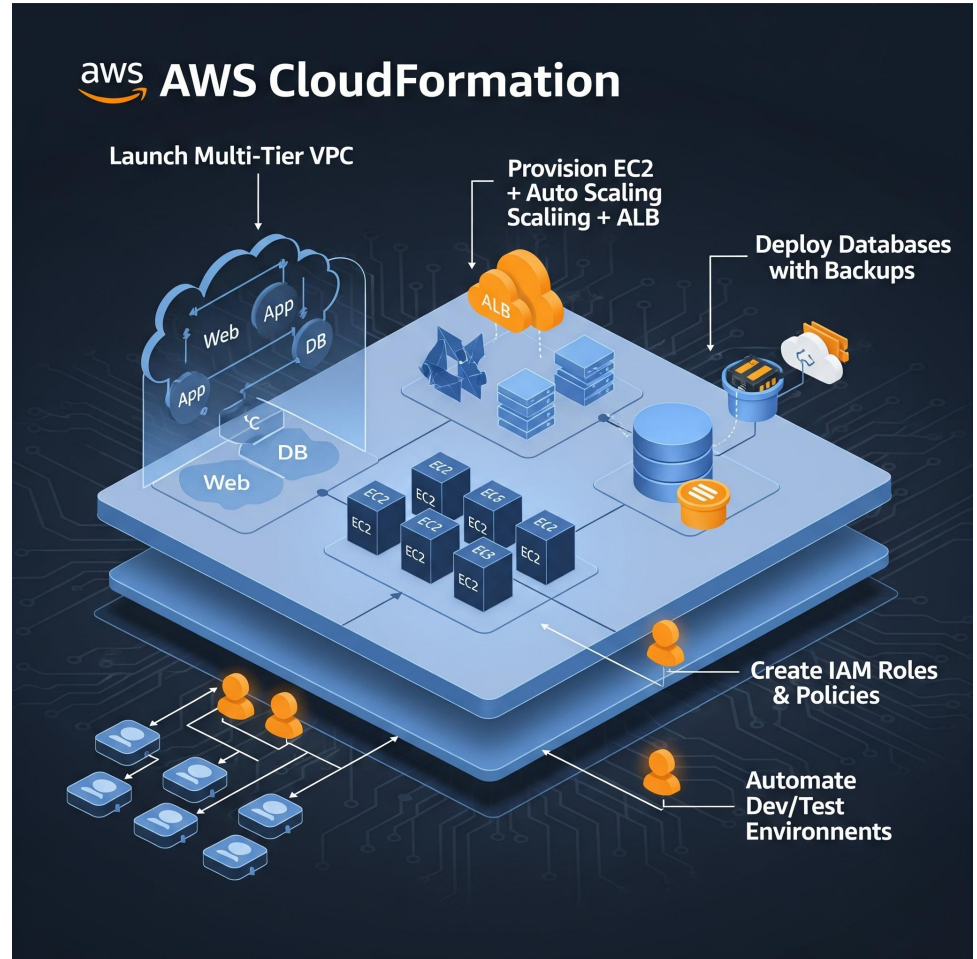
CloudFormation Building Blocks

- ◆ **Template** – blueprint for resources
- ◆ **Stack** – deployed set of resources
- ◆ **Parameters** – customize inputs
- ◆ **Outputs** – share values with other stacks
- ◆ **Resources** – AWS services defined



Common Use Cases

- ❖ Launch multi-tier VPC networks
- ❖ Provision EC2 + Auto Scaling + ALB
- ❖ Deploy databases with backups
- ❖ Create IAM roles & policies
- ❖ Automate dev/test environments



Templates

Template Structure Overview

- ❖ AWSTemplateFormatVersion
- ❖ Description
- ❖ Parameters
- ❖ Resources (mandatory)
- ❖ Outputs

AWSTemplateFormatVersion: "2010-09-09"

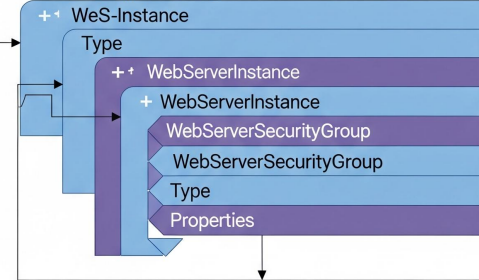
Description

A sample CloudFormation template

Parameters

	Type	Value
InstanceType	Default	Value
KeyName.....		Value
KeyName	Default	

Resources | mandatory*

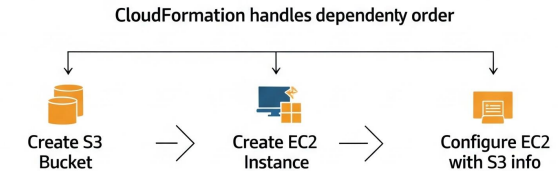
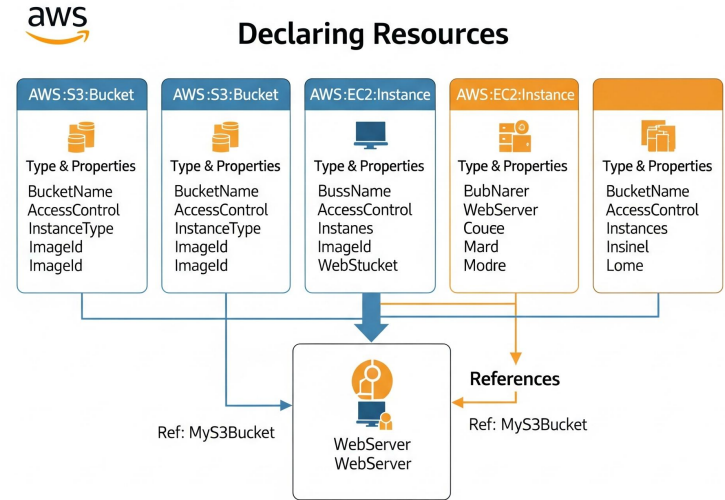


Outputs

PublicDNS	Description
PublicIP	Desperties
PublicIP	

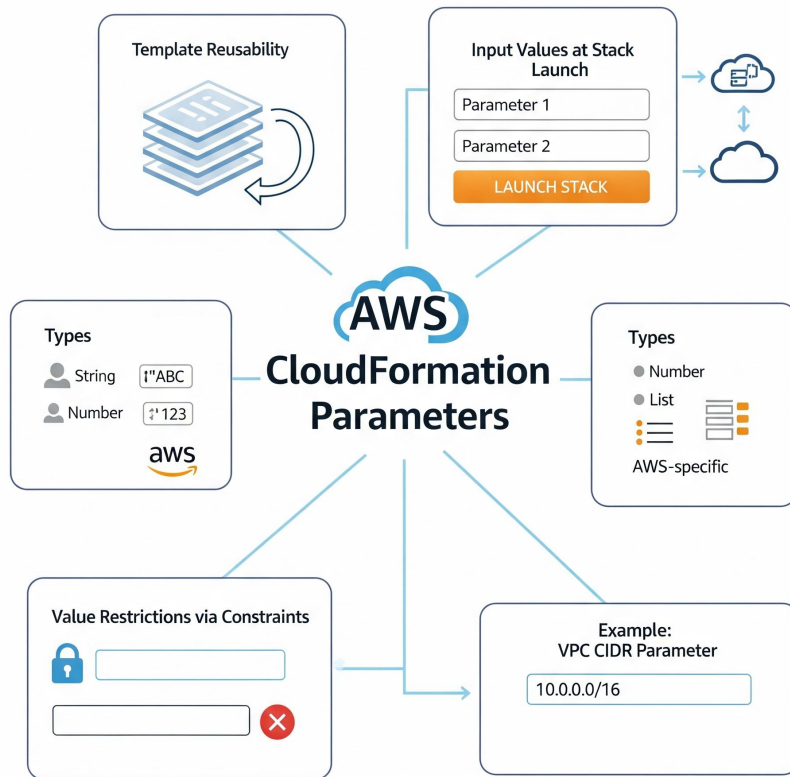
Declaring Resources

- ❖ Each resource has **Type** & **Properties**
- ❖ Example: **AWS::S3::Bucket**
- ❖ Names can be logical IDs
- ❖ References connect resources
- ❖ CloudFormation handles dependency order



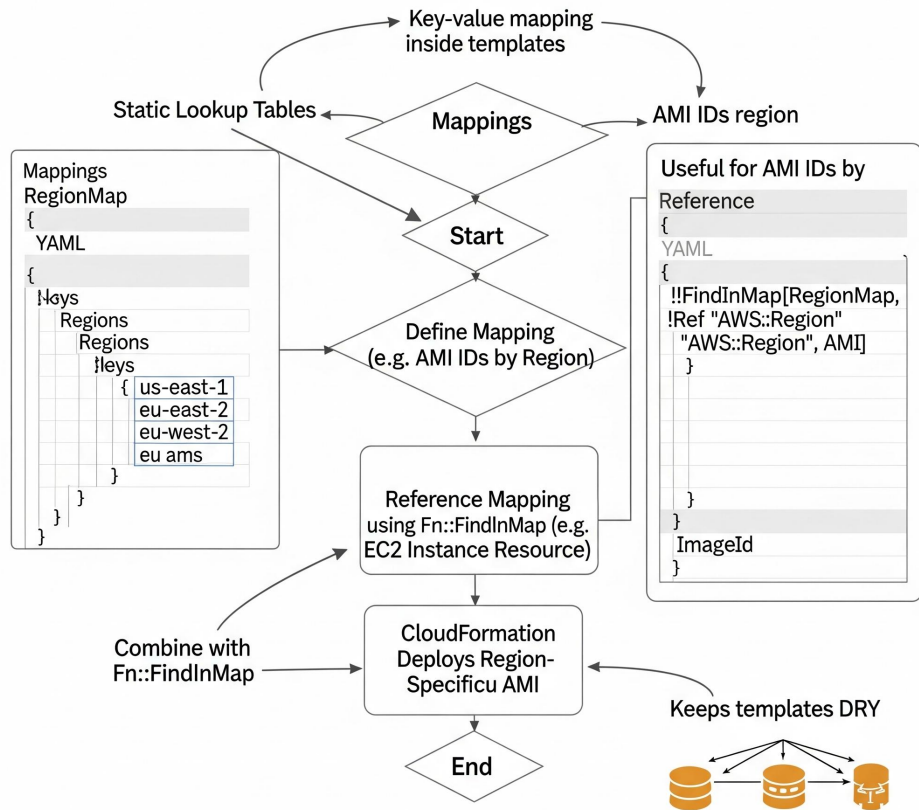
Parameters

- ❖ Enable template reusability
- ❖ Accept input values at stack launch
- ❖ Types: String, Number, List, AWS-specific
- ❖ Can restrict values via constraints
- ❖ Example: VPC CIDR as parameter



Mappings

- ❖ Static lookup tables
- ❖ Key-value mapping inside templates
- ❖ Useful for AMI IDs by region
- ❖ Combine with `Fn::FindInMap`
- ❖ Keeps templates DRY



Conditions

- ❖ Create resources only if condition true
- ❖ Support **Equals**, **Not**, **And**, **Or**
- ❖ Example: create NAT only if parameter = "Prod"
- ❖ Reduces unnecessary resources
- ❖ Adds flexibility

aws CloudFormation Conditions

Create resources only if conditions



Support

Equals

And

Or

Not

Or



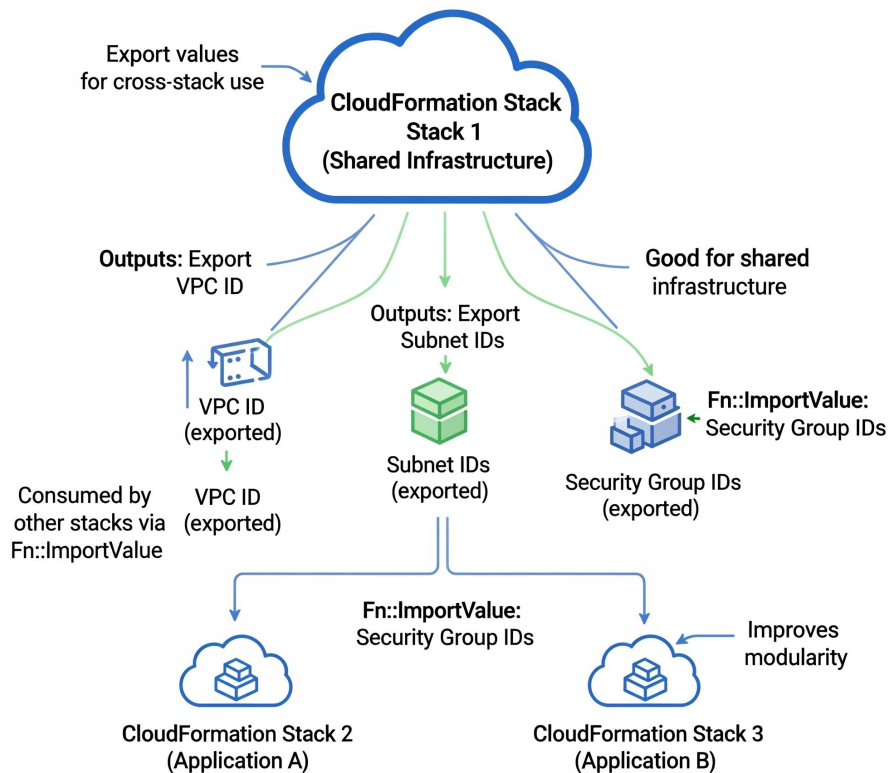
Example: NAT only if
parameter = "Prod"



Reduces unnecessary
parameters = "Prod"

Outputs

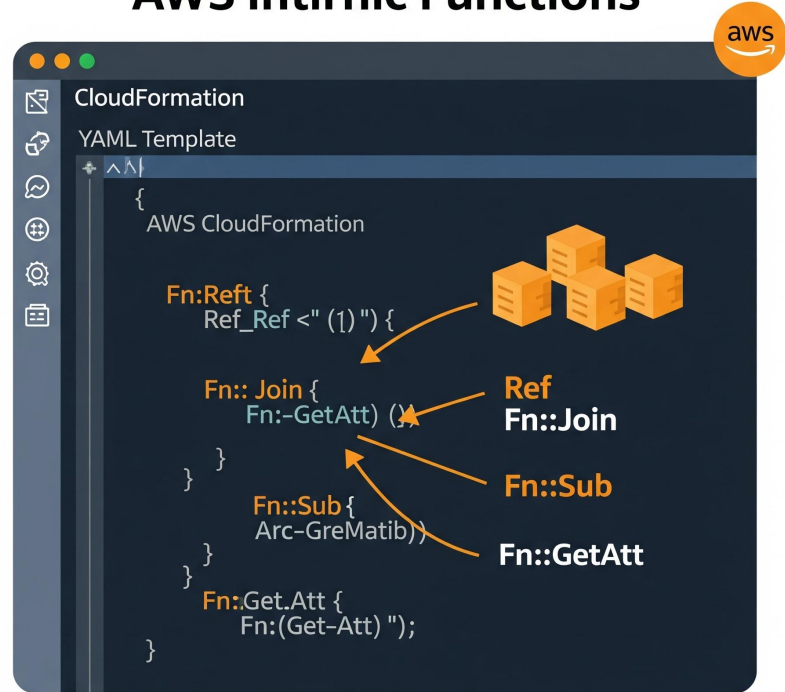
- ❖ Export values for cross-stack use
- ❖ Example: VPC ID, Subnet IDs
- ❖ Consumed by other stacks via `Fn::ImportValue`
- ❖ Good for shared infrastructure
- ❖ Improves modularity



Intrinsic Functions I

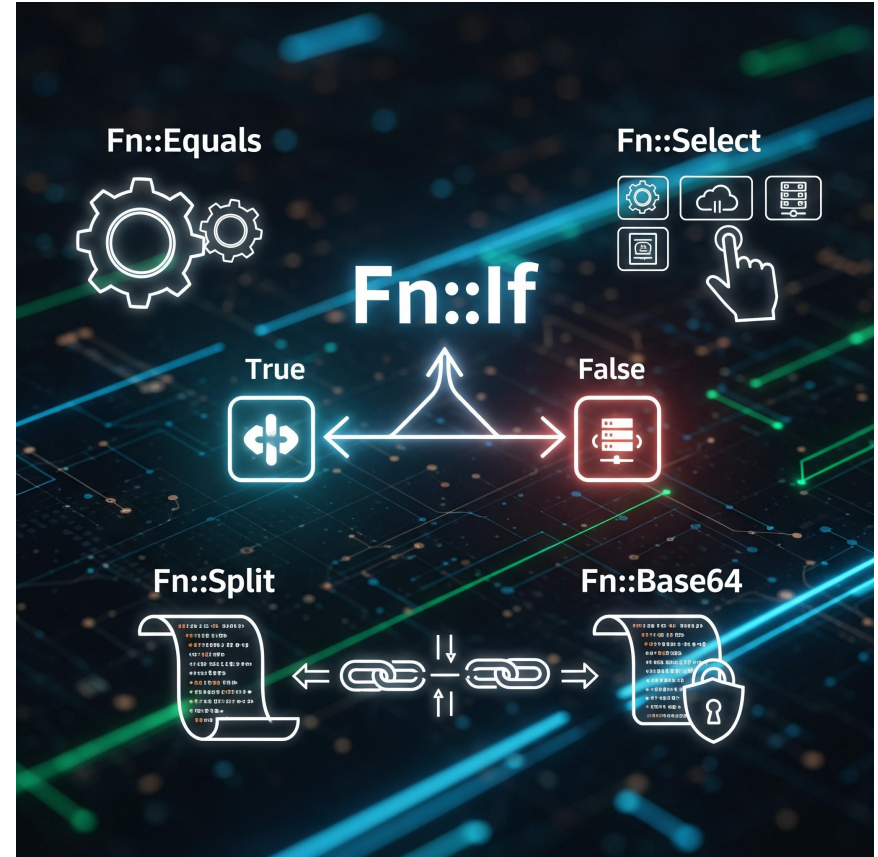
- ❖ **Ref** → reference parameter/resource
- ❖ **Fn::Join** → concatenate strings
- ❖ **Fn::Sub** → variable substitution
- ❖ **Fn::GetAtt** → resource attribute
- ❖ Widely used in templates

AWS Intirnic Functions



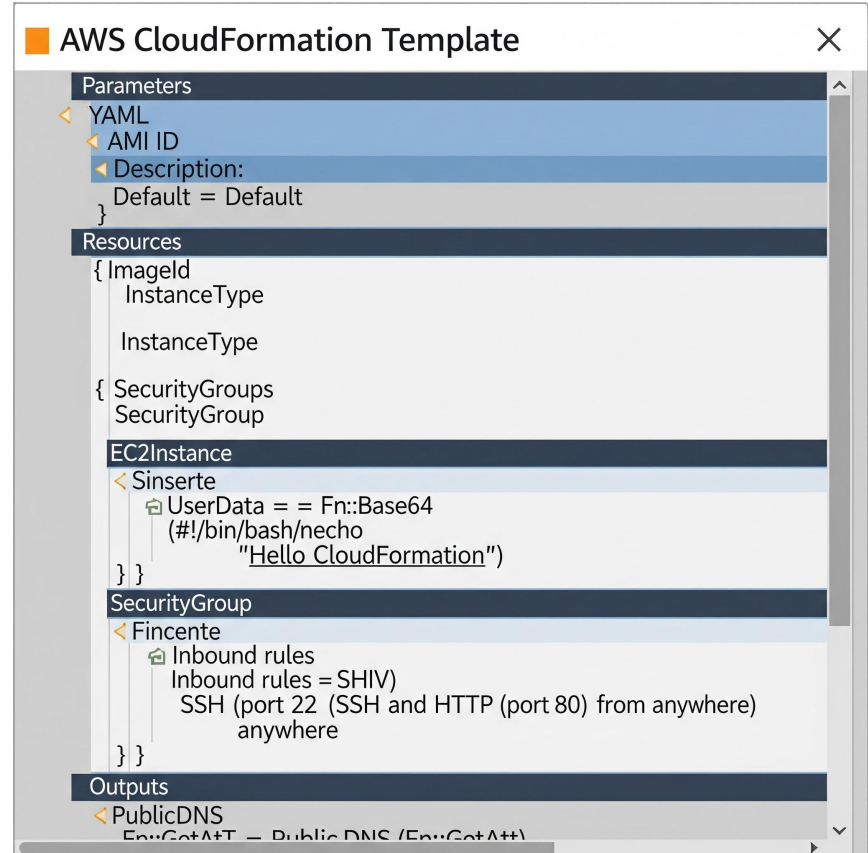
Intrinsic Function 2

- ❖ `Fn::If` → conditional logic
- ❖ `Fn::Equals`
- ❖ `Fn::Select`
- ❖ `Fn::Split`
- ❖ `Fn::Base64` (for user data scripts)



Example EC2 Template Snippet

- ❖ Parameters: AMI ID, InstanceType
- ❖ Resources: EC2 instance + SecurityGroup
- ❖ UserData encoded with **Fn::Base64**
- ❖ Output: Public DNS
- ❖ Demonstrates basics



YAML vs JSON Templates

- ❖ JSON → strict but verbose
- ❖ YAML → human-friendly, supports comments
- ❖ YAML supports shorthand for functions
- ❖ Both supported equally
- ❖ Teams prefer YAML for readability

Strict & Verbose: JSON

```
"<configuration template>"
Mvnrcfgurie() /on teneoisurise
template>
{
  configuration template)
  chagrim,ten();
  tnrrion (
    oueffion"tentiter o(loan/es"
    "agneoten()
    <laurrlrermor()()
    txigte aalg ;
    rien,astane()
    configuration_rhin))
    "c16loetilpf[
    "umtawfion is ranmer suz}

    {ste_sar"}
    loader(int.etapi)
    ratted_is))
    ganigd_ib_oiong-son))
    Joén.s(1-conigunfasaapply")})

  };
}
```

Human-Friendly: YAML

```
YAML
sand int-yiin();
incton:
  epanhe #het"lge()"
  +/;
  xpetrer sst;
  "commeond'fiispn-ter)){
  //{
  "hye- sxant";
  :lotupe(
    "ne opres sighe"yam()")
    //
    "pecterapele();
    "racee- int ai.uptupi"--());
  };
};
```

☒

JSON

☒

YAML

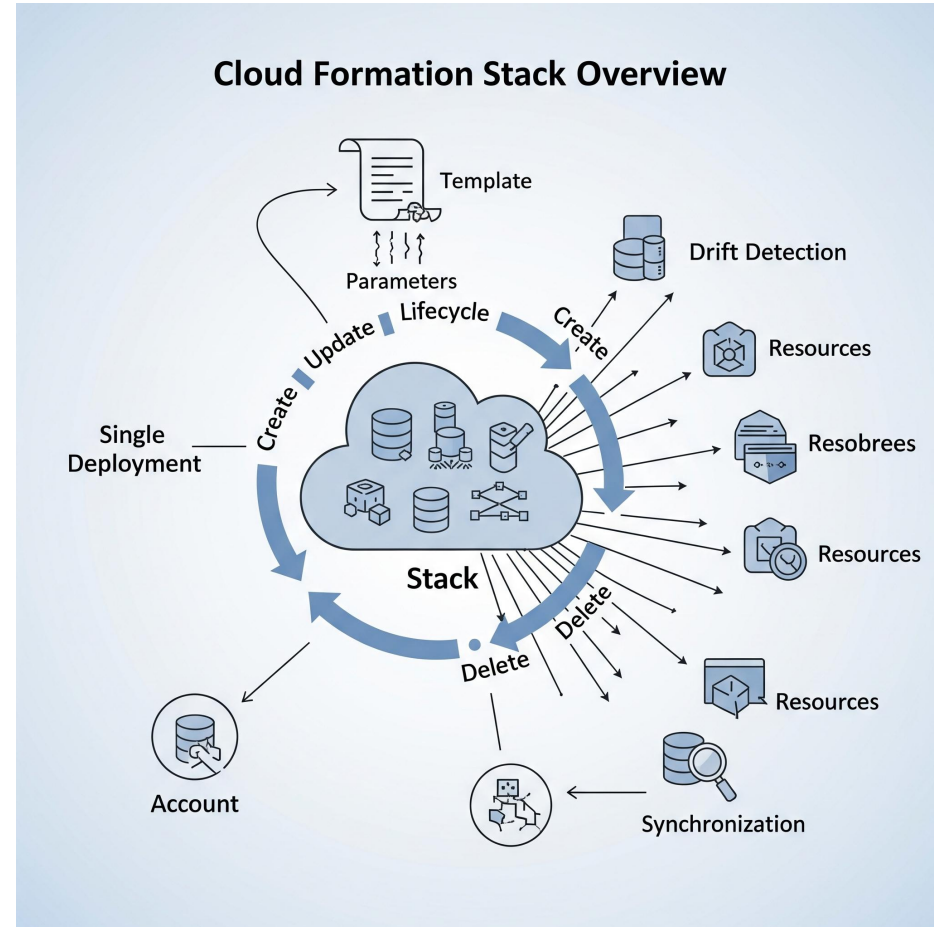
☒ EQUALLY SUPPORTED

YAML Preferred for Readability

Stack and Updates

Stack Overview

- ❖ Single deployment of resources
- ❖ Template + parameters → stack
- ❖ Lifecycle: Create → Update → Delete
- ❖ Resources tied to stack's lifecycle
- ❖ Drift detection ensures sync



Stack Creation

- ❖ Open CFN console
- ❖ Upload template file or S3 URL
- ❖ Provide parameters
- ❖ Review IAM capabilities
- ❖ Create → AWS provisions resources

The screenshot displays the AWS CloudFormation console interface. The left sidebar contains navigation icons for Home, Recent, My CloudFormation, My CloudTrail, My CloudWatch, My CloudFormation StackSets, My CloudFormation StackPolicies, My CloudFormation StackDrift, My CloudFormation StackSets, My CloudFormation StackPolicies, and My CloudFormation StackDrift. The main content area is titled 'Create stack' and features a 'Create stack' button highlighted with an orange border. Below this, there are sections for 'Example' (showing an AWS CloudFormation template), 'Specify stack details' (with fields for Stack name, Region, and Template URL), 'Configure stack options' (with a section for IAM capabilities), and a 'Review' section at the bottom right. The 'Review' section shows the stack name 'AWS-CloudFormation-Stack' and a warning message: 'AWS CloudFormation stack may create additional resources'. A 'Create stack' button is located at the bottom right of the console.

Create stack

Upload a template file

template.yaml

Amazon S3 URL

http://s3.amazonaws.com/...

Specify stack details

Stack name

Region

Template URL

Configure stack options

IAM capabilities

☐ I acknowledge that AWS CloudFormation might create IAM resources

Review

AWS CloudFormation stack may create additional resources

Create stack

Stack Creation

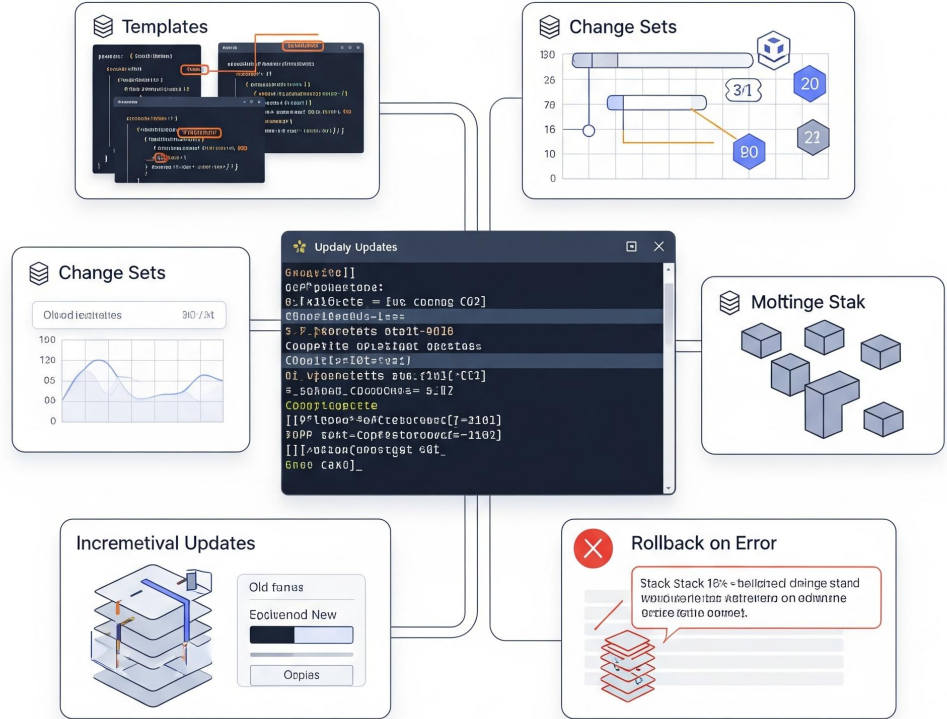
- ❖ Command: `aws cloudformation create-stack`
- ❖ `--template-body` or `--template-url`
- ❖ `--parameters` for inputs
- ❖ Supports `--capabilities` for IAM roles
- ❖ Useful for automation

```
aws cloudFormation create-stack
```

```
aws cloudFormation create MyStack  
  --template-body file://mytemplate.yaml  
  --template-url https://mybucket.s3.amazonaws.com/mytemplate.js  
  --parameters //mybucket.s3.amazonaws.com/mytemplate.json  
  
  --parameters ParametrKy=MyParam,ParanneValue=MyValue  
  
  --capabilities  
    CAPABILITY_IAM CAPABILITY_NAMED_IAM
```

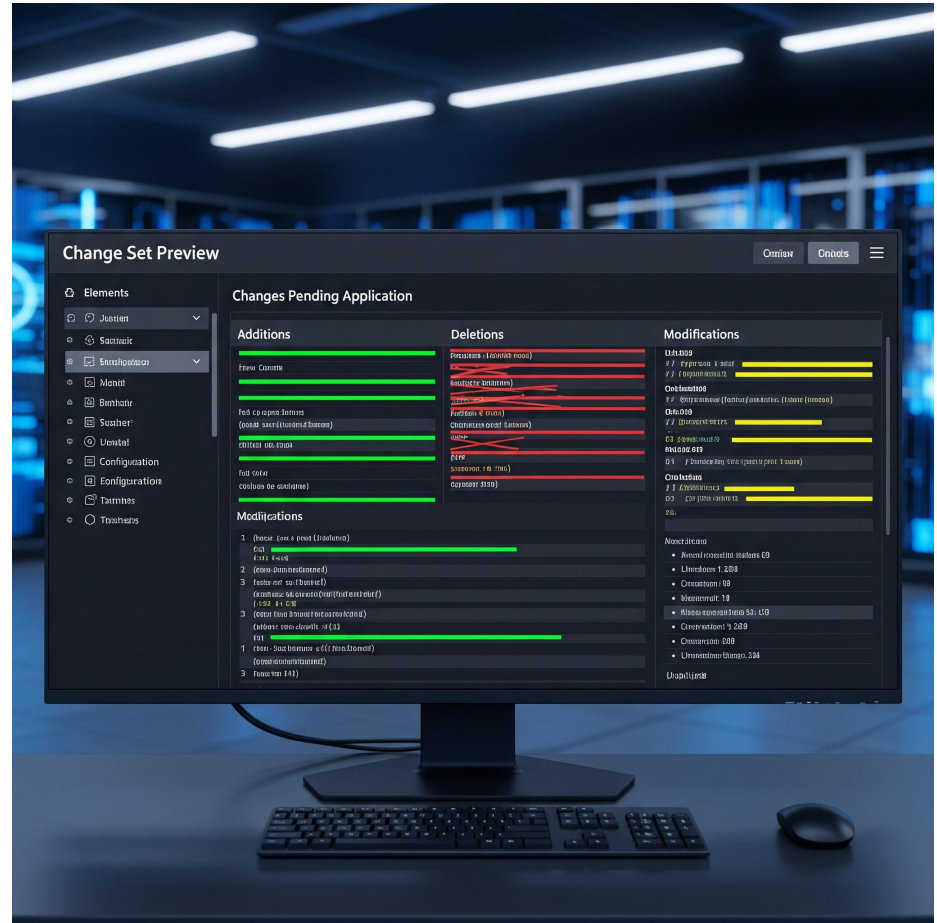
Updating Stack

- ❖ Modify template or parameters
- ❖ Apply via console or CLI
- ❖ CFN calculates change set
- ❖ Apply updates incrementally
- ❖ Rollback if errors occur



Change Sets

- ❖ Preview changes before applying
- ❖ List adds, deletes, modifies
- ❖ Prevents accidental downtime
- ❖ Approvals possible before execute
- ❖ Best practice for production



Stack Policies

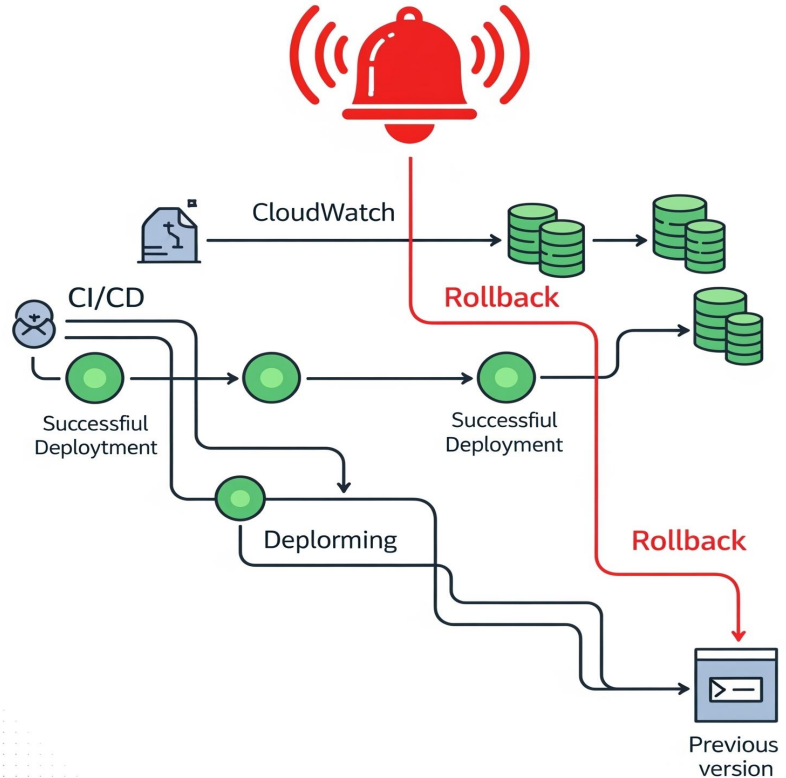
- ❖ JSON doc attached to stack
- ❖ Prevent accidental updates/deletes
- ❖ Protect critical resources
- ❖ Example: block DB deletion
- ❖ Helps enforce compliance



Rollback Triggers

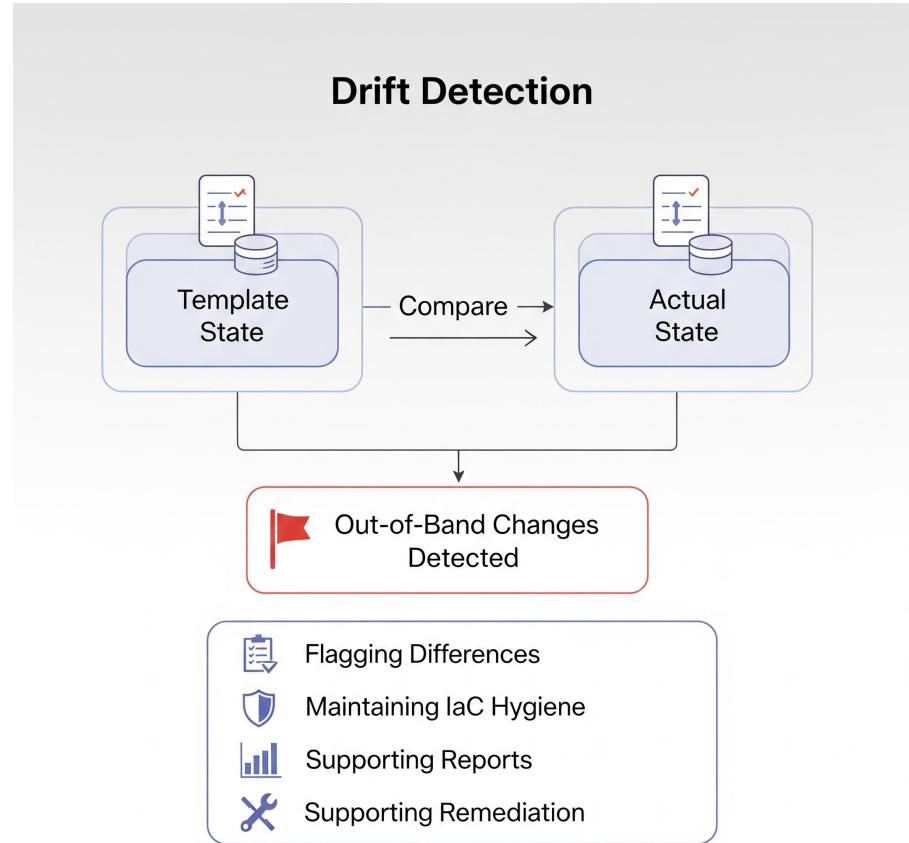
- ❖ Specify CloudWatch alarms
- ❖ If alarm breaches, rollback triggered
- ❖ Ensures safe deployments
- ❖ Automated safety net
- ❖ Reduces risk

Automatce Safety Net



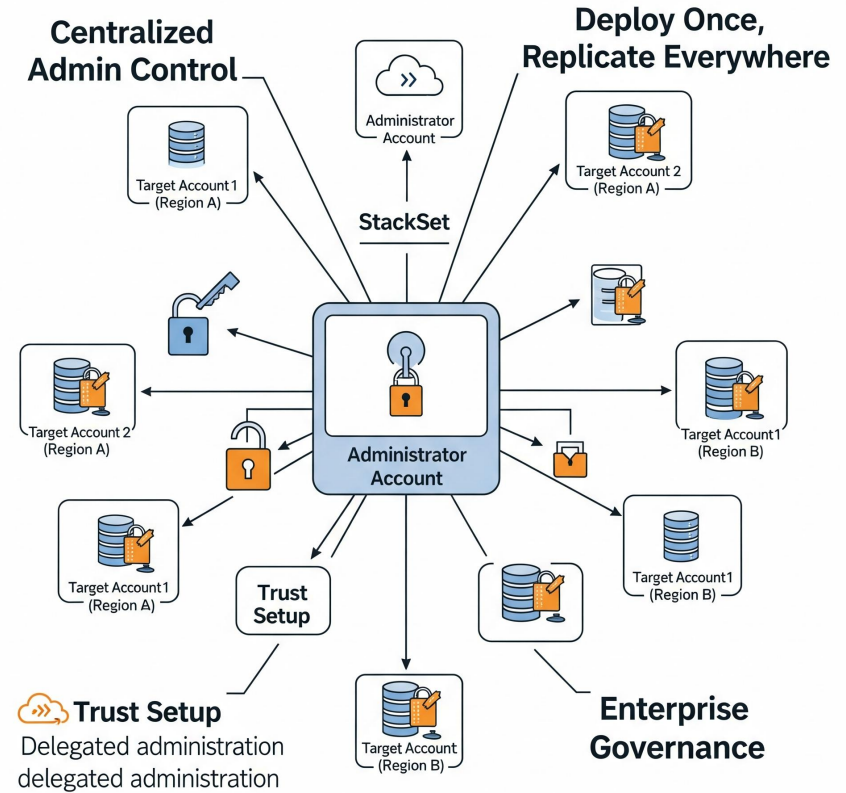
Drift Detection

- ❖ Detects out-of-band changes
- ❖ Compares actual vs template state
- ❖ Flags differences in properties
- ❖ Helps maintain IaC hygiene
- ❖ Supports reports & remediation



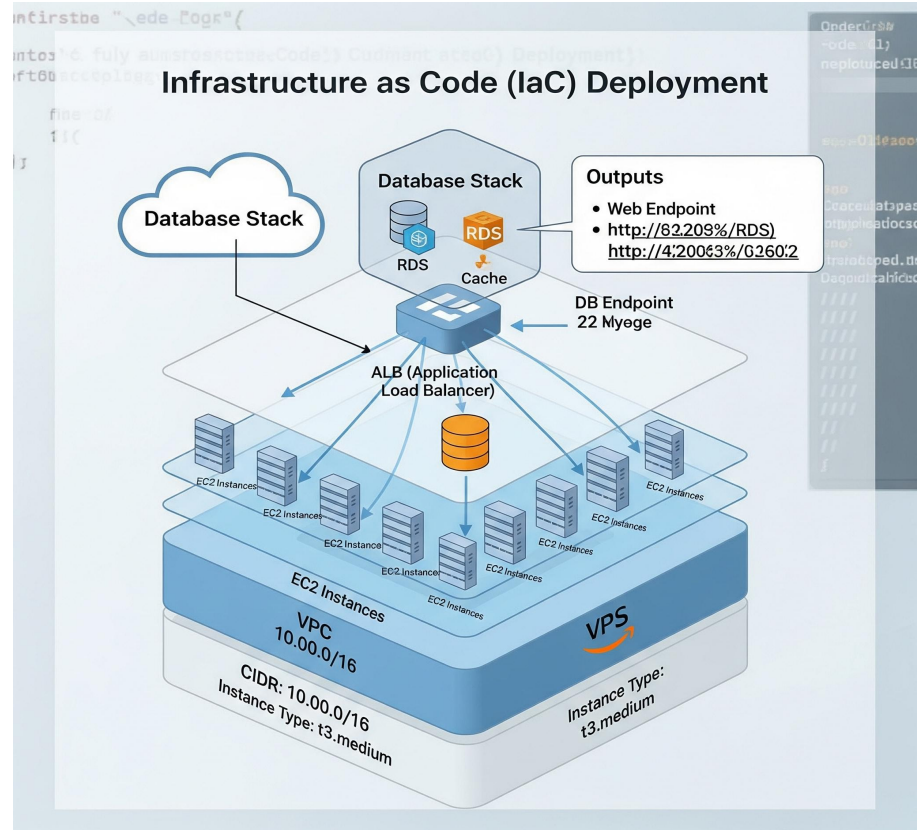
StackSets

- ❖ Manage stacks across accounts/regions
- ❖ Centralized admin account control
- ❖ Deploy once → replicate everywhere
- ❖ Great for enterprise governance
- ❖ Requires trust setup (delegated admin)



Example Scenario-Web Application Development

- ❖ Parameters: CIDR, instance type
- ❖ Resources: VPC, EC2, ALB
- ❖ Nested stack for database
- ❖ Outputs for endpoints
- ❖ Full IaC deployment



AWS CloudFormation-Example

Template

AWS::Template::FormatVersion: '2010-09-09'

Description: Create an encrypted S3 bucket with versioning

Parameters:

BucketName:

Type: String

Description: Globally unique bucket name

Resources:

Bucket:

Type: AWS::S3::Bucket

Properties:

BucketName: !Ref BucketName

VersioningConfiguration: { Status: Enabled }

BucketEncryption:

ServerSideEncryptionConfiguration:

- ServerSideEncryptionByDefault: { SSEAlgorithm:

AES256 }

Outputs:

BucketNameOut:

Value: !Ref Bucket

Export: { Name: !Sub '\${AWS::StackName}-BucketName' }

Stack Creation (Console)

- ❖ CloudFormation → **Create stack** → **With new resources (standard)**
- ❖ Template file: upload `s3-basic.yaml`
- ❖ Parameters → `BucketName=your-unique-name-12345`
- ❖ Next → acknowledge → **Create stack**

Stack Creation (CLI)

```
aws cloudformation create-stack \  
  --stack-name s3-basic \  
  --template-body file:///s3-basic.yaml \  
  --parameters
```

```
ParameterKey=BucketName,ParameterValue=your-unique-name-12345
```

```
aws cloudformation wait stack-create-complete --stack-name s3-basic
```

Update (enable public Access)

Add to Properties

```
PublicAccessBlockConfiguration:  
  BlockPublicAcls: true  
  BlockPublicPolicy: true  
  IgnorePublicAcls: true  
  RestrictPublicBuckets: true
```

Then run:

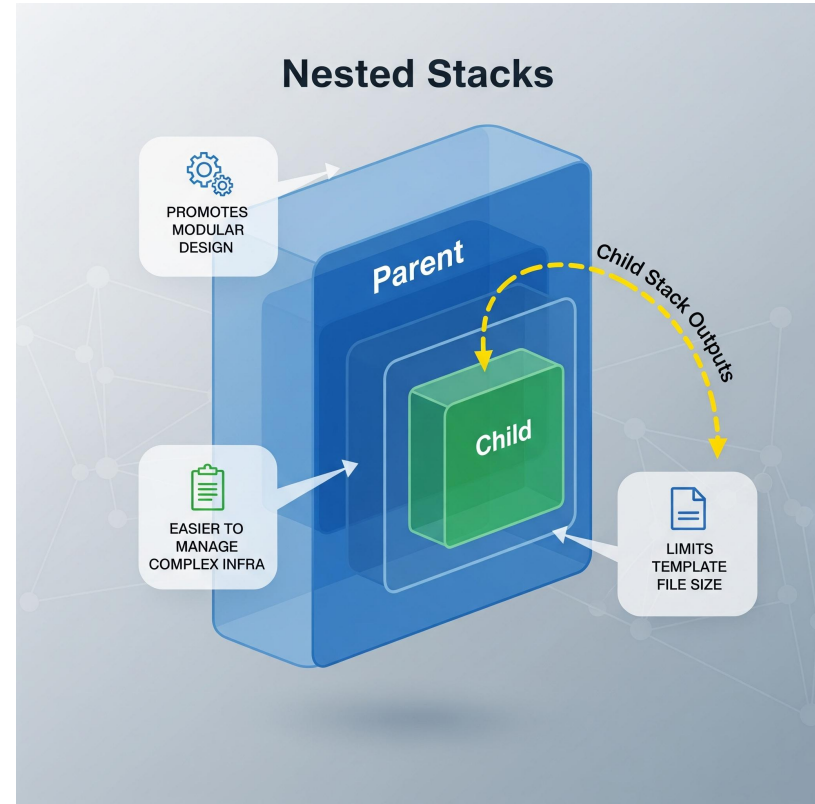
```
aws cloudformation update-stack \  
  --stack-name s3-basic \  
  --template-body file:///s3-basic.yaml
```

```
aws cloudformation wait stack-update-complete --stack-name s3-basic
```

Nested Stack and Advance concept

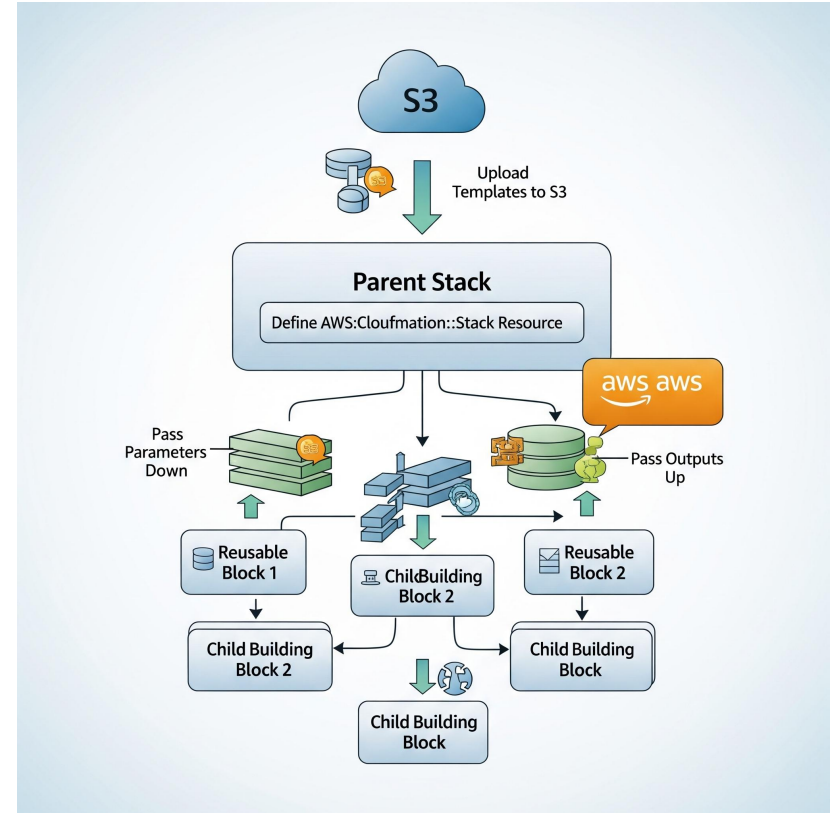
Nested Stacks Concept

- ❖ Stack inside another stack
- ❖ Reference child stack outputs
- ❖ Promotes modular design
- ❖ Easier to manage complex infra
- ❖ Limits template file size



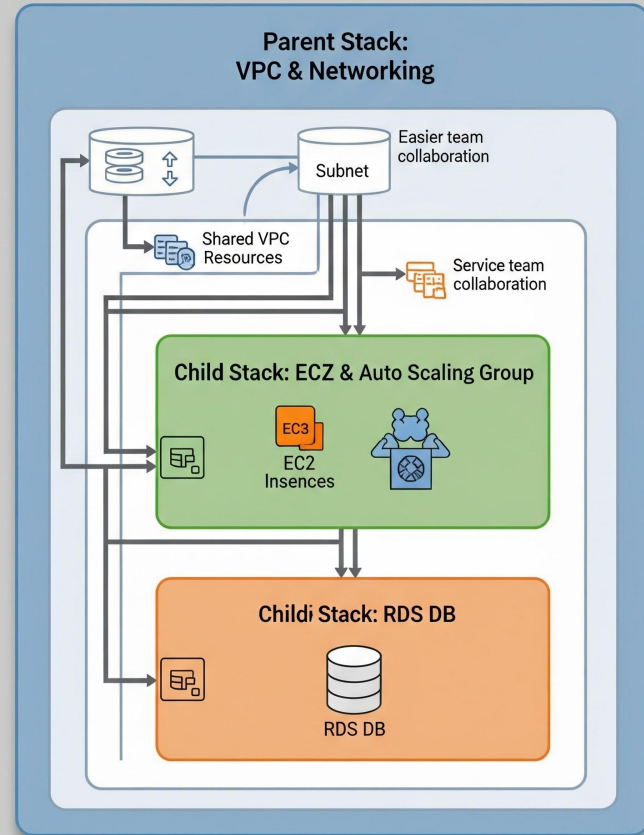
Creating Nested Stacks

- ❖ Upload child templates to S3
- ❖ In parent template:
`AWS::CloudFormation::Stack`
- ❖ Parameters passed down to child
- ❖ Outputs passed up
- ❖ Reusable building blocks



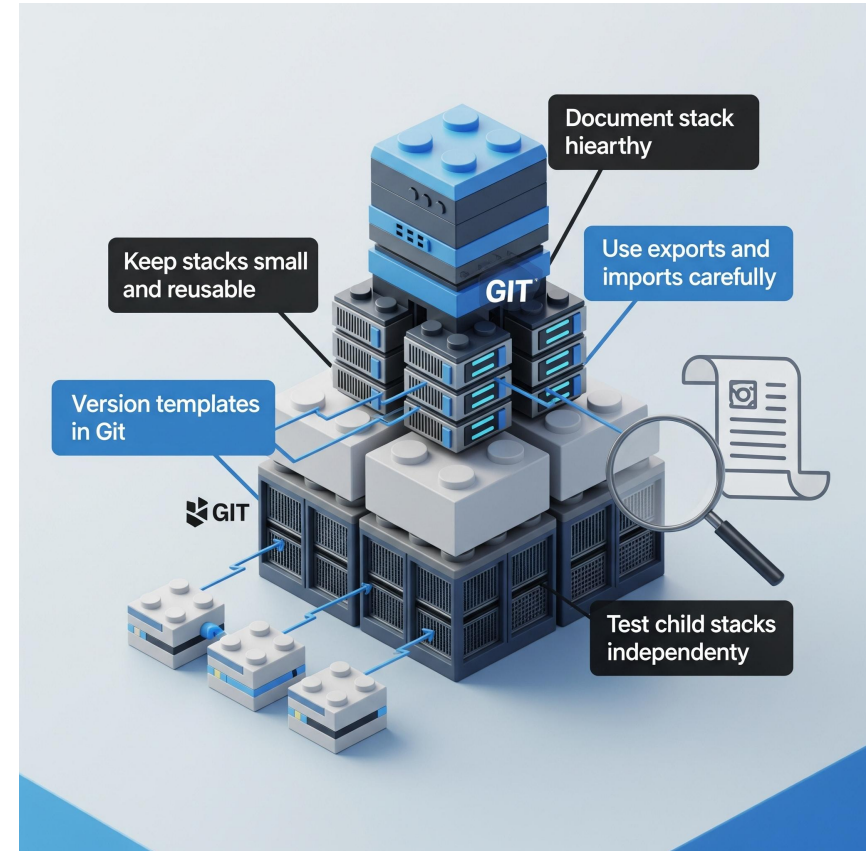
Example Nested Stack

- ❖ Parent: VPC & networking
- ❖ Child: EC2 & Auto Scaling group
- ❖ Another Child: RDS DB
- ❖ Parent ties everything together
- ❖ Easier team collaboration



Best Practices Nested Stacks

- ❖ Keep stacks small & reusable
- ❖ Version templates in Git
- ❖ Use exports/imports carefully
- ❖ Document stack hierarchy
- ❖ Test child stacks independently



Parameters Best Practises

- ❖ Provide defaults where possible
- ❖ Use AllowedValues for constraints
- ❖ Secure strings for sensitive values
- ❖ Use descriptive names & help text
- ❖ Avoid too many parameters (complexity)

CloudFormation Parameters: Best Practices



Provide Defaults

-Pre-filled

How parameter is on station and parameter



Use AllowedValues for Constraints

Parameter



Secure Sensitive Values

Use bilwet tive centroneting wate ont poficiutis tace and sorweler prelp



Avoid Too Many Parameters
Balanced balances and ws the,mtversnd of esper parameters



Use AllowedValues

Use Parameter

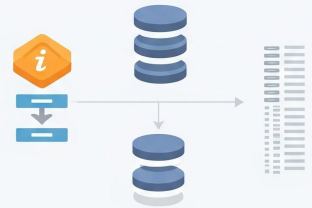
Nonups

Canay



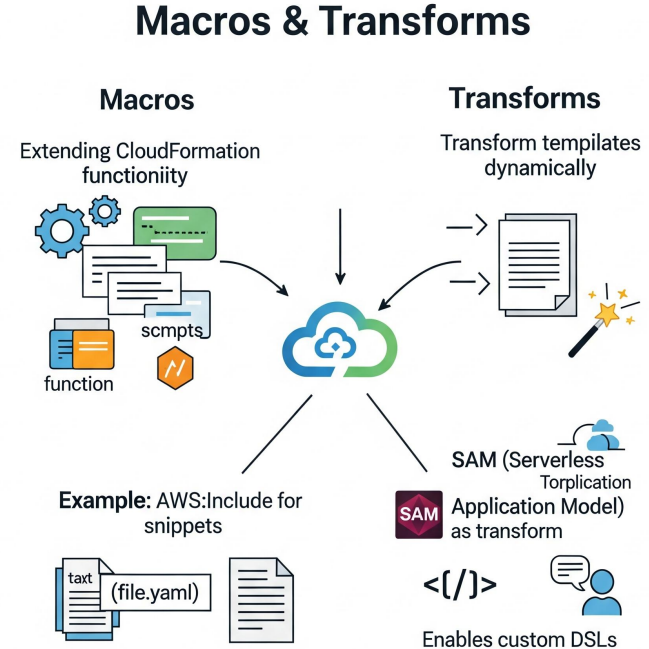
Use Descriptive Names & Help Text

Help toxmests



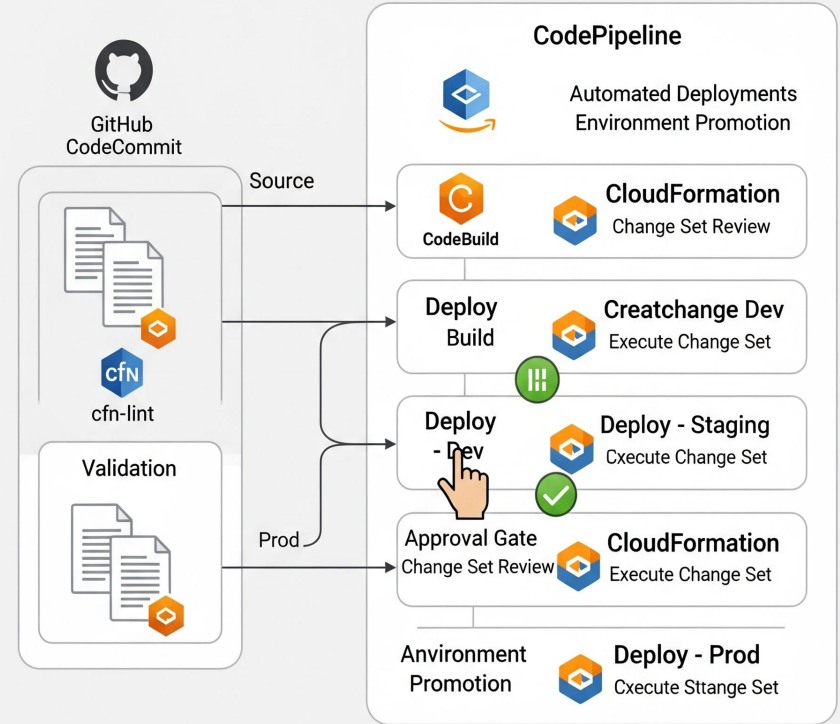
Macros and Transforms

- ❖ Extend CloudFormation functionality
- ❖ Transform templates dynamically
- ❖ Example: AWS::Include for snippets
- ❖ SAM (Serverless Application Model) as transform
- ❖ Enables custom DSLs



CloudFormation with CI/CD

- ❖ Store templates in GitHub/CodeCommit
- ❖ Validate with `cfn-lint` in pipeline
- ❖ Use CodePipeline + CodeBuild to deploy
- ❖ Change sets as approval gates
- ❖ Automate across environments

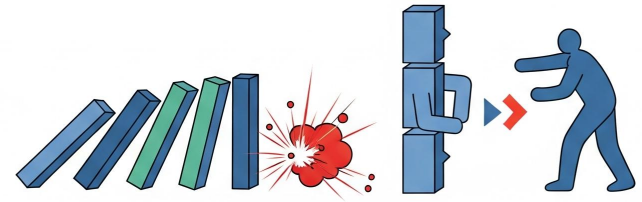


CloudFormation Limitation

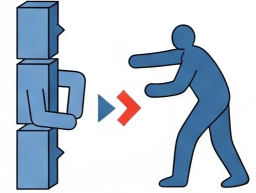
- ❖ Steeper learning curve than console
- ❖ Errors can cascade on rollback
- ❖ Less flexible than CDK for devs
- ❖ JSON/YAML debugging needed
- ❖ Still central for compliance



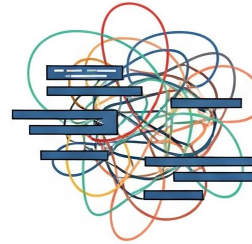
Steeper Learning Curve



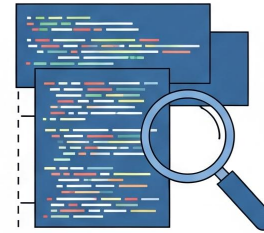
Errors Cascade on Rollback



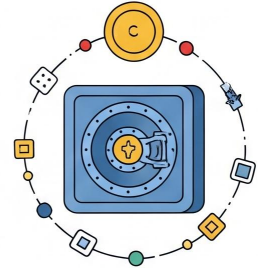
Less Flexible than CDK



JSON/YAML Debugging



JSON/YAML

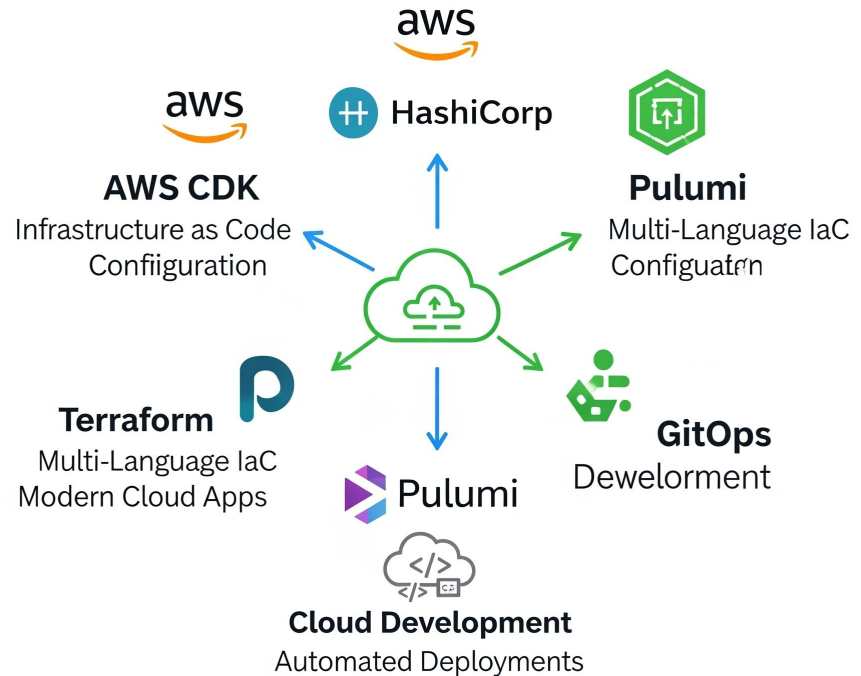


Still Central for Compliance

CloudFormation Alternatives

- ❖ AWS CDK (higher-level constructs)
- ❖ Terraform (multi-cloud IaC)
- ❖ Pulumi (IaC in programming languages)
- ❖ Cloud Development + GitOps
- ❖ Choose based on team skills

CloudFormation Alternatives



Key Takeaways

- ❖ CloudFormation = AWS native IaC
- ❖ Templates define resources declaratively
- ❖ Stacks manage lifecycle consistently
- ❖ Nested stacks improve modularity
- ❖ Integrates with CI/CD & governance

